

LRU Cache - Complete Tutorial

Last Updated : 27 Dec, 2024



What is LRU Cache?

Cache replacement algorithms are efficiently designed to replace the cache when the space is full. The **Least Recently Used (LRU)** is one of those algorithms. As the name suggests when the cache memory is full, LRU picks the data that is least recently used and **removes** it in order to make space for the new data. The priority of the data in the cache changes according to the need of that data i.e. if some data is fetched or updated recently then the priority of that data would be changed and assigned to the **highest priority**, and the priority of the data **decreases** if it remains unused operations after operations.

Operations on LRU Cache:

- **LRUCache (Capacity c):** Initialize LRU cache with positive size capacity **c**.
- **get (key) :** Returns the value of key '**k**' if it is present in the cache otherwise it returns -1. Also updates the priority of data in the LRU cache.
- **put (key, value):** Update the value of the key if that key exists, Otherwise, add a **key-value** pair to the cache. If the number of keys exceeds the capacity of the LRU cache then dismiss the least recently used key.

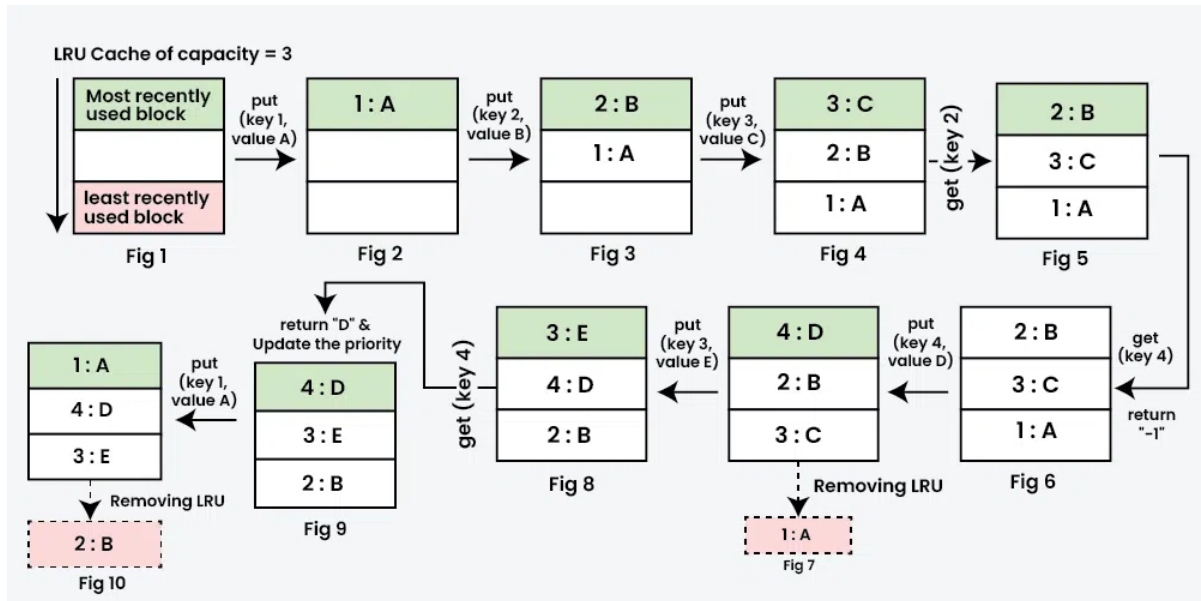
Working of LRU Cache:

Let's suppose we have an LRU cache of **capacity 3**, and we would like to perform the following operations:

- **put (key=1, value=A)** into the cache
- **put (key=2, value=B)** into the cache
- **put (key=3, value=C)** into the cache
- **get (key=2)** from the cache
- **get (key=4)** from the cache
- **put (key=4, value=D)** into the cache

- **put (key=3, value=E)** into the cache
- **get (key=4)** from the cache
- **put (key=1, value=A)** into the cache

The above operations are performed one after another as shown in the image below:



Detailed Explanation of each operation:

1. **put (key 1, value A)** : Since LRU cache has empty capacity=3, there is no need for any replacement and we put {1 : A} at the top i.e. {1 : A} has highest priority.
2. **put (key 2, value B)** : Since LRU cache has empty capacity=2, again there is no need for any replacement, but now the {2 : B} has the highest priority and priority of {1 : A} decrease.
3. **put (key 3, value C)** : Still there is 1 empty space vacant in the cache, therefore put {3 : C} without any replacement, notice now the cache is full and the current order of priority from highest to lowest is {3:C}, {2:B}, {1:A}.
4. **get (key 2)** : Now, return value of key=2 during this operation, also since key=2 is used, now the new priority order is {2:B}, {3:C}, {1:A}
5. **get (key 4)**: Observe that key 4 is not present in the cache, we return '-1' for this operation.
6. **put (key 4, value D)** : Observe that cache is FULL, now use LRU algorithm to determine which key is least recently used. Since {1:A} had the least priority, remove {1:A} from our cache and put {4:D} into the cache. Notice that the new priority order is {4:D}, {2:B}, {3:C}

7. put (key 3, value E) : Since key=3 was already present in the cache having value=C, so this operation won't result in removal of any key, rather it will update the value of key=3 to ' E' . Now, the new order of priority will become {3:E}, {4:D}, {2:B}
8. get (key 4) : Return the value of key=4. Now, new priority will become {4:D}, {3:E}, {2:B}
9. put (key 1, value A) : Since our cache is FULL, so use our LRU algorithm to determine which key was least recently used, and since {2:B} had the least priority, remove {2:B} from our cache and put {1:A} into the cache. Now, the new priority order is {1:A}, {4:D}, {3:E}.

How to design your own LRU Cache?

Input: [*LRUCache* cache = new *LRUCache* (2) , put(1 ,1) , put(2 ,2) , get(1) , put(3 ,3) , get(2) , put(4 ,4) , get(1) , get(3) , get(4)]

Output: [1 , -1, -1, 3, 4]

Explanation: The values mentioned in the output are the values returned by get operations .

- Initialize *LRUCache* class with capacity = 2.
- **cache.put(1, 1)** : (key, pair) = (1,1) inserted and has the highest priority.
- **cache.put(2, 2)** : (key , pair) = (2,2) inserted and has the highest priority.
- **cache.get(1)** : For key 1, value is 1, so 1 returned and (1,1) moved to the highest priority.
- **cache.put(3, 3)** : Since cache is full, remove least recently used that is (2,2), (3,3) inserted with the highest priority.
- **cache.get(2)**: returns -1 (key 2 not found)
- **cache.put(4, 4)** : Since the cache is full, remove least recently used that is (1,1). (4,5) inserted with the highest priority.
- **cache.get(1)**: return -1 (not found)
- **cache.get(3)**: return 3 , (3,3) will moved to the highest priority.
- **cache.get(4)** : return 4 , (4,4) moved to the highest priority.

Thoughts about Implementation Using Arrays, Hashing and/or Heap

The idea is to implement using an array to store nodes, where each node holds a key-value pair. The primary operations, get and put, are performed with $O(n)$ time complexity due to the need to search through the array. The size of the array will be equal to the given capacity of the cache.

put(int key, int value) If the cache is full, find the node with the oldest timestamp (least recently used) and replace this node with the new key and value.

else, simply add the new node to the end of the array with the timestamp of insertion.

Time Complexity: $O(n)$ (because you might have to search for the oldest node)

get(int key) Search through the array for the node with the matching key. If found, update its timestamp and return its value, else return -1.

Time Complexity: $O(n)$ (because you might have to check every node)

Can we make both operations in $O(1)$ time? we can think of hashing. With hashing, we can insert, get and delete in $O(1)$ time, but changing priorities would take linear time. We can think of using heap along with hashing for priorities. We can find and remove the least recently used (lowest priority) in $O(\log n)$ time which is more than $O(1)$ and changing priority in the heap would also be required.

Please refer [Design LRU Cache](#) for details of all approaches.

Efficient Solution - Using Doubly Linked List and Hashing

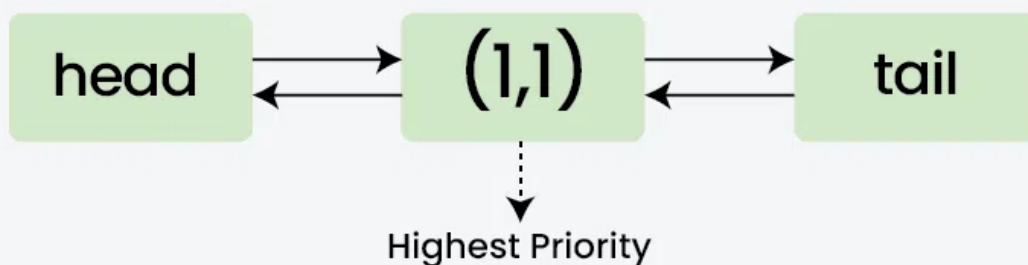
The basic idea behind implementing an **LRU (Least Recently Used)** cache using a **key-value** pair approach is to manage **element access** and **removal efficiently** through a combination of a **doubly linked list** and a **hash map**.

- When adding a new **key-value** pair, insert it as a **new node** at the **head** of the **doubly linked list**. This ensures that the **newly added key-value pair** is marked as the **most recently used**.
- If the key is already **present** in the **cache**, get the **corresponding node** in the doubly linked list using **hashmap**, **update** its value and move it to the head of the list, and **update** its **position** in the **hashmap** also. This operation ensures that the accessed **key-value** pair is considered the **most recently used**.

- The priority of nodes in the **doubly linked list** is based on their **distance** from the **head** . **Key-value** pairs closer to the head are **more recently used** and thus have **higher priority** . Also, key-value pairs closer to the **tail** are considered **less recently used** and have **lower priority**.
- When the **cache** reaches its **maximum capacity** and a **new key-value pair** needs to be added, **remove** the node from **hashmap** as well as from the **tail** in the **doubly linked list** . **Tail** node represents the **least recently used key-value pair** and is **removed** to make **space** for the **new entry**.

02
Step

put (1,1): Since (key, value) = (1,1) is not present and cache is not full, we can put (1,1) after the head.



Doubly Linked List Representation Of LRU Cache



2 / 10

Please refer [LRU cache implementation using Doubly Linked List and Hashing](#) for details

Complexity Analysis of the Efficient Solution

- **Time Complexity:**
 - put() operation: $O(1)$ i.e. time required to insert or update new key-value pair is constant

- `get()` operation: $O(1)$ i.e. time required to get the value of a key is constant

- **Auxiliary Space:** $O(c)$ where c is the capacity of the Cache.

Advantages of LRU cache:

- Accessing data is done in $O(1)$ time
- Updating a key-value pair also takes $O(1)$ time.
- Removing the least recently used item is performed in $O(1)$ time.
- LRU plays a crucial role in page replacement strategies to minimize [page faults](#).

Disadvantages of LRU cache:

- It requires large cache size to increase efficiency.
- It requires additional Data Structure to be implemented.
- In LRU, error detection is difficult as compared to other algorithms.
- It has limited acceptability.

Real-World Application of LRU Cache:

- In Database Systems for fast query results.
- In Operating Systems to minimize page faults.
- Text Editors and IDEs to store frequently used files or code snippets.
- Network routers and switches use LRU to increase the efficiency of computer network.
- In compiler optimizations.
- Text Prediction and autocompletion tools.

Complete Tutorial on LRU Cache with Implementations

 Comment

More info 

Advertise with us

Next Article 

Detect cycle in an undirected graph
using BFS

Similar Reads

Queue Data Structure

A Queue Data Structure is a fundamental concept in computer science used for storing and managing data in a specific order. It follows the principle of "First in, First out" (FIFO), where the first element added...

🕒 2 min read

Introduction to Queue Data Structure

Queue is a linear data structure that follows FIFO (First In First Out) Principle, so the first element inserted is the first to be popped out. FIFO Principle in Queue: FIFO Principle states that the first element...

🕒 5 min read

Introduction and Array Implementation of Queue

Similar to Stack, Queue is a linear data structure that follows a particular order in which the operations are performed for storing data. The order is First In First Out (FIFO). One can imagine a queue as a line o...

🕒 2 min read

Queue - Linked List Implementation

In this article, the Linked List implementation of the queue data structure is discussed and implemented. Print '-1' if the queue is empty. Approach: To solve the problem follow the below idea: we maintain two...

🕒 8 min read

Applications, Advantages and Disadvantages of Queue

A Queue is a linear data structure. This data structure follows a particular order in which the operations are performed. The order is First In First Out (FIFO). It means that the element that is inserted first in the...

🕒 5 min read

Different Types of Queues and its Applications

Introduction : A Queue is a linear structure that follows a particular order in which the operations are performed. The order is First In First Out (FIFO). A good example of a queue is any queue of consumers...

🕒 8 min read

Queue implementation in different languages



Some question related to Queue implementation



Implementation of Deque using doubly linked list

A Deque (Double-Ended Queue) is a data structure that allows adding and removing elements from both the front and rear ends. Using a doubly linked list to implement a deque makes these operations...

🕒 9 min read

Queue using Stacks

Given a stack that supports push and pop operations, your task is to implement a queue using one or more instances of that stack along with its operations. [Table of Content](#) [By Making Enqueue Operation...](#)

🕒 11 min read

implement k Queues in a single array

Given an array of size n, the task is to implement k queues using the array. `enqueue(qn, x)` : Adds the element x into the queue number qn `dequeue(qn, x)` : Removes the front element from queue number qn...

🕒 15+ min read

LRU Cache - Complete Tutorial

What is LRU Cache? Cache replacement algorithms are efficiently designed to replace the cache when the space is full. The Least Recently Used (LRU) is one of those algorithms. As the name suggests whe...

🕒 8 min read

Easy problems on Queue



Intermediate problems on Queue

